Sprint 2 – Week 2 Progress Report

SWENG 894 Capstone

MicroMatch

GitHub Repository Link

Chastidy Joanem

1. System Overview and Context

1.1 System Scope

MicroMatch is a web-based micro-internship marketplace designed to connect students with short-term, skill-based professional opportunities. The platform enables:

- Organizations to post structured, well-defined micro-projects.
- Students to browse and discover opportunities.
- Students to apply for projects.
- Organizations to manage and review applications.
- A controlled workflow to manage project participation.

The system architecture follows a layered full-stack model:

- **Frontend:** React-based SPA (Single Page Application)
- **Backend:** FastAPI REST API
- **Database:** PostgreSQL (relational persistence)
- **Authentication:** JWT-based stateless authentication
- **Authorization:** Role-Based Access Control (RBAC)

The platform supports two primary user roles:

- Student
- Organization

1.2 Sprint 2 Context

Sprint 2 expands the lifecycle of an application from:

- ➢ Submission → Visibility → Decision → Status Transition

Week 1 implemented:

- Application submission (US-6)
- Student-side status visibility (US-8)

Week 2 extends functionality to:

- Organization-side application review (US-7)

- Accept/Reject decision workflow (US-9)
- Frontend organization dashboard integration

1.3 Functional Requirements to User Stories Mapping

Functional Requirement	User Story
FR-1: The system shall allow users to create an account.	US-1
FR-2: The system shall support role-based access control.	US-1
FR-3: The system shall allow students to create/edit profile.	US-2
FR-4: The system shall allow organizations to create profile.	US-3
FR-5: The system shall allow orgs to post listings.	US-4
FR-6: The system shall allow students to browse/search listings.	US-5
FR-7: The system shall allow students to apply.	US-6
FR-8: The system shall track application status.	US-8
FR-9: The system shall allow orgs to accept/reject.	US-9
FR-10: The system shall allow students to submit deliverables.	US-10
FR-11: The system shall allow orgs to review deliverables.	US-11
FR-12: The system shall allow marking projects complete.	US-12
FR-13: The system shall allow feedback submission.	US-13
FR-14: The system shall allow admin moderation.	US-14

FR-15: The system shall log system actions.	US-15

This traceability ensures alignment between formal IEEE-style functional requirements and Agile user stories used for sprint planning.

2. Sprint 2 – Week 2 Overview

2.1 Sprint Goal

The goal of Sprint 2 – Week 2 is:

- To implement the Organization Review Workflow, enabling organizations to:
  - View applications submitted to their projects
  - Accept or reject applicants
  - Enforce proper state transitions
  - Reflect status updates consistently across the system

This sprint transforms the platform from passive application tracking into an active decision-based workflow system.

3. Sprint Planning

3.1 Groomed Product Backlog

Priority Definitions

- High: Core MVP functionality required for end-to-end workflow.
- Medium: Important workflow enhancements dependent on core features.
- Low: Administrative or cross-cutting features not required for initial MVP validation.

ID	User Story	Priority	Story Points	Sprint	Rationale
US-1	Registration/Login	High	5	Sprint 1	Required for all access
US-2	Student Profile	High	5	Sprint 1	Core student data
US-3	Org Profile	High	5	Sprint 1	Required before posting

US-4	Post Listing	High	8	Sprint 1	Core marketplace feature
US-5	Browse/Search	High	5	Sprint 1	Enables discovery
US-6	Apply	High	5	Sprint 2	Core interaction
US-7	Review Applications	Medium	5	Sprint 2	Depends on applications
US-8	Track Status	High	3	Sprint 2	Enhances UX
US-9	Accept/Reject	Medium	3	Sprint 2	Workflow control
US-10	Submit Deliverables	Medium	5	Sprint 3	Completion flow
US-11	Review Deliverables	Medium	3	Sprint 3	Completion validation
US-12	Mark Complete	Medium	3	Sprint 3	Project closure
US-13	Feedback	Low	5	Sprint 3	Post-completion feature
US-14	Admin Moderation	Low	5	Sprint 3	Governance feature
US-15	Logging	Low	8	Sprint 3	Cross-cutting concern

3.2 Sprint Backlog – Week 2 Commitment

Week 2 focuses on organization-side workflow logic and controlled state transitions within the application lifecycle.

User Story	Description	Story Points
US-7	Organization View Applications	5
US-9	Accept / Reject Applications	5
Supporting Tasks	Dashboard Integration + Validation	3

Total Commitment: 13 Story Points

The sprint commitment was intentionally scoped to the organization-side review workflow to avoid overlapping development with completion-phase features (e.g., deliverables or feedback).

By isolating visibility and decision logic in Week 2, the sprint ensures:

- Clean separation between submission (Week 1) and decision-making (Week 2)
- Controlled implementation of state transitions
- Reduced integration risk across roles
- Clear validation of ownership and authorization constraints

This phased rollout continues the incremental workflow expansion strategy established in Week 1 and aligns with Agile best practices for layered feature delivery.

Capacity Planning and Velocity Justification

As a single-developer team, historical velocity data indicates:

- Sprint 1 weekly average: ~15–18 story points
- Sprint 2 Week 1 completed: 17 story points

Week 2 includes:

- Backend endpoint implementation (2 major endpoints)
- Ownership validation logic
- Pagination support
- Status transition enforcement
- Role-based access control validation
- Frontend dashboard integration
- Confirmation modal UX implementation
- Unit + system test coverage (≥95%)

Unlike Week 1, this sprint does not introduce a new database model. However, it introduces:

- Business rule enforcement (state transitions)
- Authorization complexity (project ownership validation)
- Cross-role data exposure controls
- Bidirectional frontend/backend synchronization

Given these factors, a commitment of 13 story points was determined to be:

- Slightly lighter than Week 1 to account for increased logic complexity
- Focused on correctness over volume
- Within sustainable velocity range

- Balanced between backend decision logic and frontend usability refinement

The reduction from 17 → 13 story points reflects intentional risk management rather than reduced productivity. Accept/reject workflows introduce higher correctness sensitivity because improper implementation can result in inconsistent system state.

By constraining scope to:

- Application visibility (US-7)
- Decision workflow (US-9)

The sprint minimizes downstream refactor risk before moving into completion-phase features in Week 3.

This represents deliberate, risk-aware sprint planning aligned with incremental delivery principles and validated historical velocity data.

3.4 Sprint 2 Backlog Structure

Sprint 2 was divided into three weekly development phases.

Week 1 – Application Creation (Student Side)

Primary Focus:

- Allow students to apply to projects
- Provide application visibility
- Prevent duplicate submissions
- Integrate frontend UI workflow

Completed Issues

Issue	Description	Story Points	Status
#45	Create Application Database Model	3	Done
#46	Implement Apply to Project Endpoint	5	Done
#47	Implement Get My Applications Endpoint	3	Done
#49	Frontend Apply Button Integration	3	Done
#50	Student Applications Dashboard	3	Done

Week 1 Total Commitment: 17 Story Points

Week 2 – Organization Review Workflow

Primary Focus:

- Application visibility for organizations
- Status decision workflow

Completed Issues

Issue	Description	Story Points	Status
#48	Organization View Applications Endpoint	5	Done
#53	Accept / Reject Endpoint	5	Done
#54	Organization Dashboard Integration	3	Done

Week 2 Total Commitment: 13 Story Points

Week 3 — Workflow Completion & Stability

Primary Focus:

- Status synchronization
- Business rule enforcement
- Integration testing

Planned Issues

Issue	Description	Story Points
#56	Status Badge Sync	3
#55	Prevent Multiple Accepted Applications	3
#57	End-to-End Workflow Testing	5

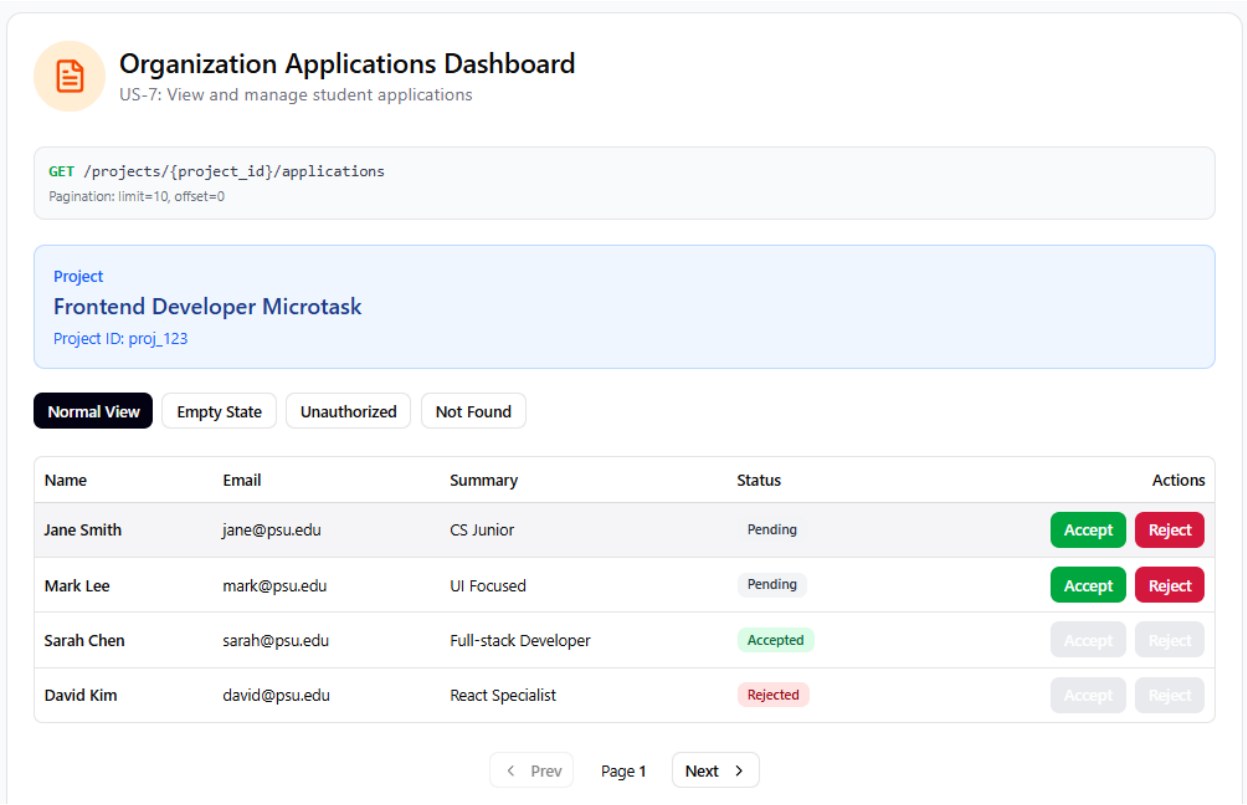
Week 3 Total Commitment: 11 Story Points

4. UI Design

Wireframes were created for US-7 and US-9.

Each interface directly maps to backend endpoints and enforces strict role and ownership validation.

4.1 Organization Applications Dashboard (US-7)



Description and Design Rationale

The Organization Applications Dashboard allows authenticated organization users to view and manage student applications submitted to their projects. This page directly supports:

- US-7: Organization View Applications
- Related Issue: #48 (Organization Applications Endpoint)
- Functional Requirement: FR-8

This interface enables organizations to review applicants in a structured and paginated format, ensuring scalability and clarity as application volume increases.

Layout Structure

The interface is structured into three primary sections:

1. Project Header Section
    a. Project Title (read-only display)
    b. Optional Project Status Indicator
2. Applications Table

    a. Student Name (read-only)
    b. Email (read-only)
    c. Profile Summary (read-only snippet)
    d. Application Status (badge)
    e. Actions Column (Accept / Reject buttons)
3. Pagination Controls
    a. Previous / Next buttons
    b. Page indicator

This table-based layout ensures structured presentation of multi-record data and aligns with standard administrative dashboard patterns.

Usability Considerations

Status Badges:

- Status values (Pending, Accepted, Rejected) are visually distinguished using color-coded badges to reduce cognitive load and enable rapid scanning.

Action Button States:

- Accept/Reject buttons are disabled when status ≠ "pending" to prevent invalid transitions.

Pagination Support:

- Limit/offset-based pagination ensures scalability and prevents performance degradation when application volume grows.

Consistent Table Layout:

- Column alignment ensures readability and supports comparison between applicants.

Empty State Handling:

- If no applications exist, a clear message is displayed:
    - "No applications have been submitted for this project yet."

Validation & Error Handling

Unauthorized Access:

- If an organization attempts to access a project they do not own, access is denied and a permission error message is displayed.

Project Not Found:

- Displays a safe error message without exposing system details.

Pagination Validation:

- Invalid limit/offset parameters return appropriate validation feedback.

All error messaging is displayed inline without full page refresh to preserve user workflow continuity.

Security Alignment

- Accessible only to users with role = Organization.
- JWT authentication required.
- Ownership validation ensures organizations can only view applications for projects they created.
- Sensitive student information exposure is limited to:
  - Name
  - Email
  - Profile summary
- No confidential data is exposed beyond required fields.

Alignment With Backend Design

This UI maps directly to:

Endpoint:
GET /projects/{project_id}/applications

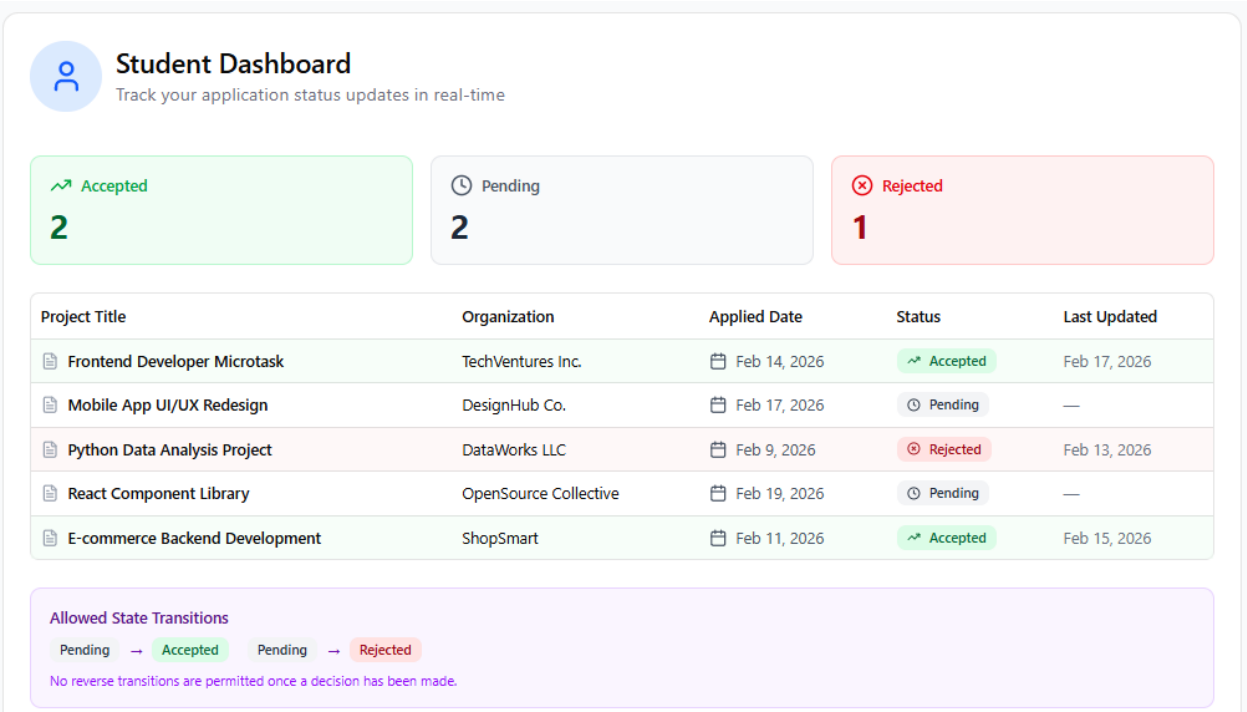
Response Schema Includes:

- student.name
- student.email

- student.profile_summary
- application.status
- Pagination metadata (limit, offset)

The wireframe reflects the one-to-many relationship between Project and Application, and enforces foreign key ownership validation.

4.2 Accept / Reject Decision Workflow (US-9)



Description and Design Rationale

The Accept / Reject Decision interface allows organizations to update the status of submitted applications. This page directly supports:

- US-9: Accept / Reject Applications
- Related Issue: #53 (Accept / Reject Endpoint)
- Functional Requirement: FR-9

This interface introduces controlled state transitions within the application lifecycle, enforcing business logic constraints and preventing inconsistent system states.

Layout Structure

The decision workflow is integrated within the Applications Dashboard and includes:

- Action Buttons (Per Application Row)
- Accept Button
- Reject Button
- Confirmation Modal
- Decision confirmation prompt
- Confirm / Cancel buttons
- Status Update Feedback
- Updated status badge displayed immediately
- Success or error notification message

This design minimizes navigation overhead by keeping decision controls contextual within each application row.

Usability Considerations

Confirmation Modal:

- Prevents accidental decisions by requiring explicit confirmation before status changes.

Immediate UI Feedback:

- Status badge updates dynamically after successful API response.

Button State Management:

- Buttons disabled once a decision is made to prevent duplicate or invalid transitions.

Visual Hierarchy:

- Accepted applications visually distinct from pending or rejected entries.

Consistency:

- Uses same badge styling as Student Dashboard to maintain system-wide uniformity.

Validation & Error Handling

Invalid Transition Handling:

- If a user attempts to update an application that is no longer in “pending” state, an error message is displayed.

Ownership Validation:

- If organization does not own the project, action is blocked and access denied.

Network/API Errors:

- Graceful fallback messaging ensures UI stability even if backend call fails.

Security Alignment

- Accessible only to role = Organization.
- Ownership validation required before status modification.
- State transitions restricted to:
  - pending → accepted
  - pending → rejected
- No reverse transitions permitted.
- JWT authentication required for all requests.

Alignment With Backend Design

Maps directly to:

Endpoint:

PATCH /applications/{application_id}/status

Accepted Request Body:

```
{ "status": "accepted" }
{ "status": "rejected" }
```

Business Logic Enforced:

- Application must belong to a project owned by the organization.
- Application must currently be in “pending” state.

This reflects controlled state management within the Application model and enforces workflow integrity across the system.

5. Acceptance Criteria

5.1 US-7 – Organization View Applications

Acceptance Criteria:

- Only authenticated organizations may access the endpoint.
- Organization must own the project.
- Project must exist.
- Endpoint must support pagination using:
  - limit
  - offset
- Response must include:
  - Student name
  - Student email
  - Student profile summary
  - Application status
  - Application date
- Response must not expose:
  - Student password
  - JWT tokens
  - Internal database identifiers unrelated to business logic
- Returns:
  - 200 on success
  - 403 if organization does not own project
  - 403 if user role is not organization
  - 404 if project not found
- Frontend Requirements:
  - Dashboard accessible only to organization users.
  - Applications displayed in table format.
  - Status displayed as badge (Pending, Accepted, Rejected).
  - Pagination controls visible when applicable.

5.2 US-9 – Accept / Reject Applications

Acceptance Criteria:

- Only authenticated organizations may update application status.
- Organization must own the project associated with the application.
- Application must exist.
- Application must currently have status = "pending".
- Valid transitions:
  - pending → accepted
  - pending → rejected
- Invalid transitions:
  - accepted → rejected
  - rejected → accepted
  - accepted → accepted
  - rejected → rejected
- System must:
  - Persist updated status in database.
  - Update updated_at timestamp.
  - Reflect status change in student dashboard.
  - Prevent multiple accepted applications if business rule enforced.
- Returns:
  - 200 on successful update
  - 400 if invalid status transition
  - 403 if user not organization
  - 403 if organization does not own project
  - 404 if application not found
- Frontend Requirements:
  - Accept/Reject buttons visible only for pending applications.
  - Confirmation modal required before decision.
  - Status badge updates immediately after decision.
  - Buttons disabled once status changes.

6. Definition of Done (Sprint 2)

The Definition of Done was refined for Sprint 2 and applied consistently to all backlog items.

A User Story is considered complete when:

Design

- API contract documented.

- State transitions defined.

Implementation

- Role-based access control enforced.
- Ownership validation implemented.
- Pagination implemented (US-7).
- Status transition logic implemented (US-9).

Testing

- Unit tests written.
- Edge case validation included.
- Minimum 3 test cases per story.
- Coverage ≥ 95%.

Integration

- Frontend integration functional
- No regression to Week 1 features
- PR merged into main branch.

7. Software Testing

7.1 Requirement-to-Test Mapping

Functional Requirement	User Story	Test Case ID
FR-8	US-7	TC-US7-01
FR-8	US-7	TC-US7-02
FR-8	US-7	TC-US7-03
FR-9	US-9	TC-US9-01
FR-9	US-9	TC-US9-02
FR-9	US-9	TC-US9-03
FR-9	US-9	TC-US9-04
FR-9	US-9	TC-US9-05

7.2 User Acceptance Test Specification

All test cases are mapped to FR-8 and FR-9, ensuring full coverage of Week 2 scope.

TC-US7-01 – Successful Application Retrieval

Related Requirement: FR-8
User Story: US-7 – Organization View Applications
Actor: Organization

Preconditions:

- Organization account exists and is authenticated.
- Organization owns the project.
- Project has one or more applications.
- Applications include varying statuses.

Steps:

1. Log in as Organization.
2. Navigate to organization project dashboard.
3. Select a specific project.
4. System sends GET /projects/{project_id}/applications?limit=10&offset=0.

Expected Result:

- HTTP 200 OK returned.
- Response payload contains paginated list.
- Each entry includes:
    - Student name
    - Student email
    - Profile summary
    - Status
    - Application date
- Pagination metadata present.
- No sensitive data exposed.
- Data matches database records exactly.

Postconditions:

- No database changes occur.
- System state remains unchanged.

TC-US7-02 – Unauthorized Organization (Not Owner)

Related Requirement: FR-8
User Story: US-7 – Organization View Applications
Actor: Organization

Preconditions:

- Organization authenticated.
- Project exists.
- Project owned by different organization.

Steps:

1. Log in as non-owning organization.
2. Send GET /projects/{project_id}/applications.

Expected Result:

- HTTP 403 Forbidden returned.
- Error message: "Not authorized to view this project's applications."
- No application data returned.

Postconditions:

- Database unchanged.
- No data exposure occurs.

TC-US7-03 – Project Not Found

Related Requirement: FR-8
User Story: US-7 – Organization View Applications
Actor: Organization

Preconditions:

- Organization authenticated.
- Project ID does not exist.

Steps:

1. Send GET request to /projects/{invalid_id}/applications.

Expected Result:

- HTTP 404 Not Found returned.
- Error message: “Project not found.”
- No data returned.

Postconditions:

- No data persisted.
- System state unchanged.

TC-US9-01 – Accept Application Successfully

Related Requirement: FR-9
User Story: US-9 – Accept / Reject
Actor: Organization

Preconditions:

- Organization authenticated.
- Organization owns project.
- Application exists.
- Application status = "pending".

Steps:

1. Navigate to organization dashboard.
2. Click “Accept” for a pending application.
3. Confirm decision in modal.
4. System sends PATCH /applications/{application_id}/status with { status: "accepted" }.

Expected Result:

- HTTP 200 OK returned.
- Application status updated to "accepted".
- updated_at timestamp modified.
- Student dashboard reflects updated status.
- Accept/Reject buttons disabled.

Postconditions:

- Database record updated.
- Only selected application modified.

TC-US9-02 – Reject Application Successfully

Related Requirement: FR-9
User Story: US-9 – Accept / Reject
Actor: Organization

Preconditions:

- Application status = "pending".

Steps:

1. Click “Reject”.
2. Confirm decision.

Expected Results:

- HTTP 200 OK returned.
- Status updated to "rejected".
- Student dashboard reflects updated status.

Postconditions:

- Application record updated.
- No other records modified.

TC-US9-03 – Invalid Status Transition

Related Requirement: FR-9
User Story: US-9 – Accept / Reject
Actor: Organization

Preconditions:

- Application status = "accepted" (or "rejected").

Steps:

1. Attempt to change status again via PATCH request.

Expected Results:

- HTTP 400 Bad Request returned.
- Error message: “Invalid status transition.”
- Status remains unchanged.

Postconditions:

- Database state preserved.
- No additional updates applied.

TC-US9-04 – Unauthorized Role Attempt

Related Requirement: FR-9
User Story: US-9 – Accept / Reject
Actor: Student

Preconditions:

- Student authenticated.
- Application exists.

Steps:

1. Send PATCH /applications/{application_id}/status.

Expected Result:

- HTTP 403 Forbidden returned.
- Error message: “Only organizations may update application status.”
- No status change.

Postconditions:

- Database unchanged.

TC-US9-05 – Application Not Found

Related Requirement: FR-9
User Story: US-9 – Accept / Reject
Actor: Organization

Preconditions:

- Organization authenticated.
- Application ID does not exist.

Steps:

1. Send PATCH request with invalid ID.

Expected Result:

- HTTP 404 Not Found returned.
- Error message: “Application not found.”
- No data modified.

Postconditions:

- System state unchanged.

8. Source Code Development Progress

8.1 Repository

GitHub Repository: https://github.com/chazdj/MicroMatch

Project board integrated with GitHub Issues for traceability between user stories and commits.

8.2 Summary of Contributions

Backend Contributions

- Implemented GET /projects/{project_id}/applications.
- Added project ownership validation.
- Enforced organization-only RBAC.
- Implemented pagination (limit/offset).
- Implemented PATCH /applications/{application_id}/status.
- Enforced valid status transitions.
- Prevented invalid state updates.

Frontend Contributions

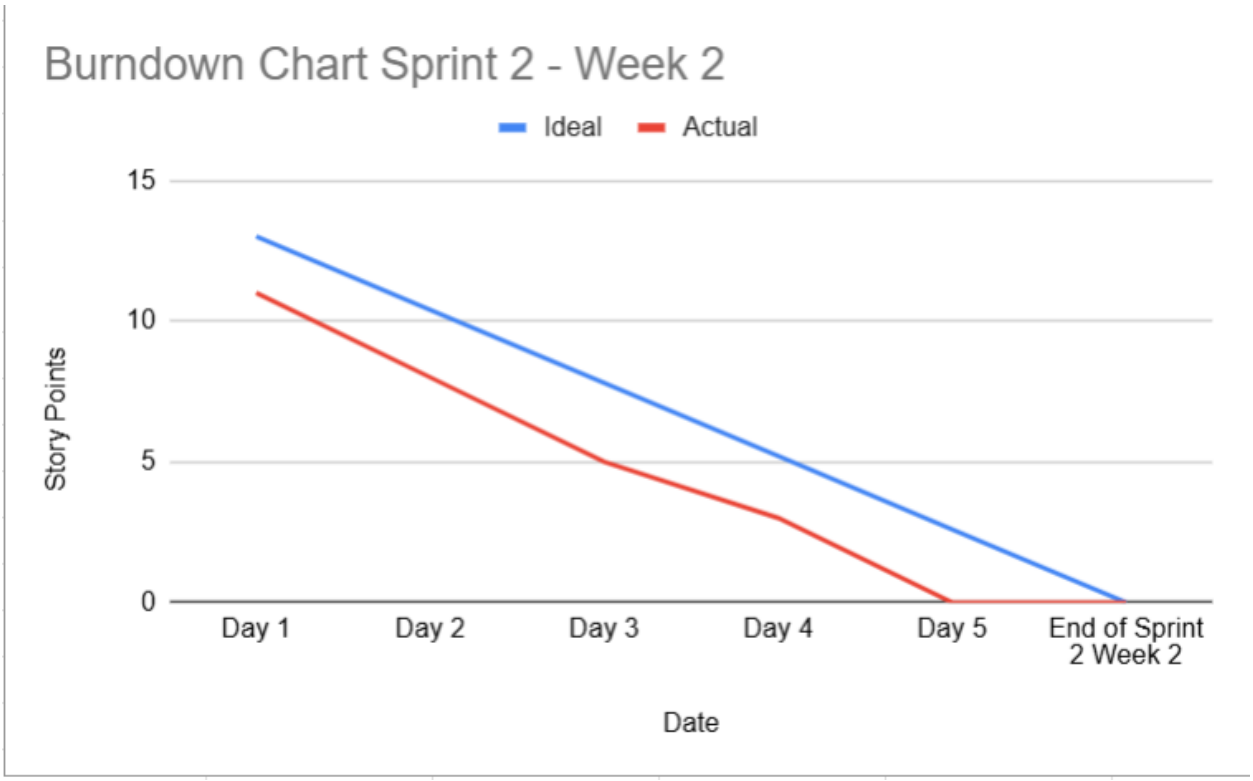
- Built Organization Applications Dashboard.
- Integrated API for applicant retrieval.

- Added pagination UI.
- Implemented Accept/Reject buttons.
- Added confirmation modal.
- Synchronized student dashboard status updates.

8.3 Important Commits

Commit Message	Related Issue	User Story
feat: org view applications endpoint	#48	US-7
test: org view applications	#48	US-7
feat: implement accept reject endpoint	#53	US-9
test: application status update	#53	US-9
test: application not found	#53	US-9
ui: organization applications dashboard	#54	US-7

8.4 Burndown Chart



Sprint Capacity: 13 Story Points

Sprint Duration: 5 Development Days

Burndown Chart Analysis

The burndown chart reflects a structured, backend-first development strategy followed by frontend integration and validation.

During Day 1, progress was incremental as foundational ownership validation and role enforcement were implemented. The modest reduction reflects architectural groundwork rather than feature completion. This aligns with risk mitigation strategy, ensuring that access control and project validation logic were stable before exposing application data.

Day 2 shows a significant decline in remaining work, corresponding to completion of the Organization View Applications endpoint and pagination logic (US-7). This milestone reduced technical uncertainty and established the data retrieval contract required for frontend integration.

Day 3 demonstrates moderate progress as core Accept/Reject logic was implemented. However, story points were not fully burned because additional validation and transition enforcement were still pending. This plateau indicates controlled pacing rather than delay.

Day 4 shows another notable reduction when full state transition validation was completed. This included:

- Preventing invalid status changes
- Enforcing ownership checks
- Ensuring only “pending” applications could transition
- Maintaining data consistency across student dashboards

This stage represented the most logic-intensive portion of the sprint.

Day 5 reflects final frontend integration, confirmation modal implementation, regression testing, and test stabilization. The sprint concludes with a clean drop to zero remaining story points, indicating full completion of committed scope.

Trend Interpretation

The burndown trend exhibits:

- No scope creep
- No mid-sprint story additions

- No upward spikes in remaining effort
- Logical clustering of backend work before UI integration

The five-day distribution shows deliberate sequencing:

1. Access control foundation
2. Data retrieval completion
3. Decision logic implementation
4. State enforcement hardening
5. UI integration and validation

This sequencing reduced integration risk and avoided rework.

Velocity Alignment

Sprint 1 average velocity: ~16 points
Sprint 2 Week 1 completed: 17 points
Sprint 2 Week 2 completed: 13 points

The slight reduction in total points this week reflects the increased complexity of state transition enforcement and validation-heavy backend logic rather than reduced productivity.

Velocity remains stable and predictable, demonstrating:

- Accurate story point estimation
- Sustainable development pacing
- Realistic sprint commitments

The sprint completed all committed work within the planned five-day window, reinforcing estimation accuracy and disciplined backlog control.

9. Backlog Grooming

During Week 2:

- No new user stories were added.
- Story point estimates remain accurate.
- No scope expansion occurred.
- Sprint 2 roadmap remains unchanged.

The backlog remains stable and aligned with sprint objectives.

10. Risk Management

Risk	Mitigation
Unauthorized project access	Ownership validation
Invalid state transitions	Controlled status logic
Data exposure risk	Limited response schema
Inconsistent UI state	Backend as single source of truth

11. Sprint 2 Week 2 Outcome

Sprint 2 Week 2 successfully delivered the Organization Review Workflow.

The system now supports:

- Organization-side application visibility
- Pagination of applicants
- Accept/reject decision capability
- State transition validation
- Full frontend/backend synchronization
- High test coverage
- Stable sprint execution

The sprint goal was fully achieved.